

RAPPORT TP 2.3 : Open VAS

INTRODUCTION

L'objectif de ce TP est de mettre en place un environnement de conteneurisation en installant Docker sur un système linux, de configurer les privilèges utilisateurs et de vérifier l'état de sécurité du système via le pare-feu (ufw)

1. Présentation des outils : Nessus / Open VAS

Nessus

Nessus est un outil propriétaire de scan de vulnérabilités utilisé en entreprise. Il permet :

- Détection rapide des failles
- Interface intuitive
- Base de vulnérabilités très riche

Open VAS

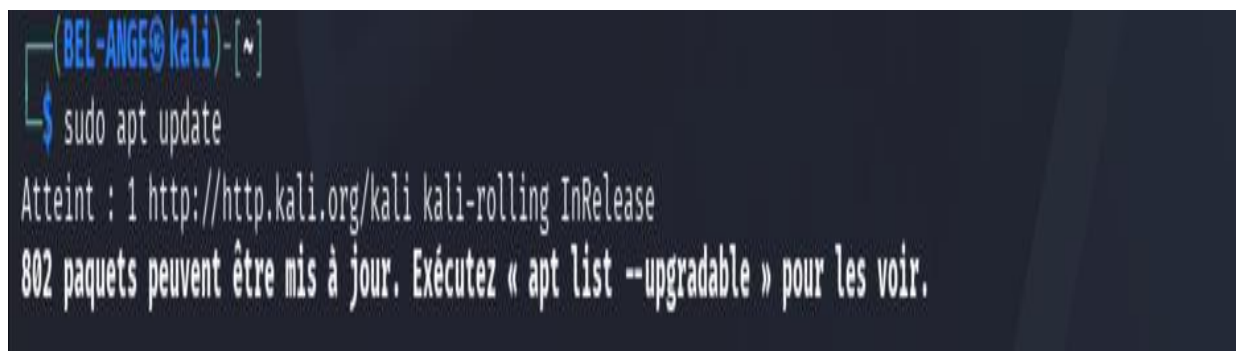
Open VAS est une alternative open-source à Nessus :

- Gratuit
- Très complet
- Plus technique à configurer

2. Résultats scans (AVEC CAPTURES) : Linux / Windows

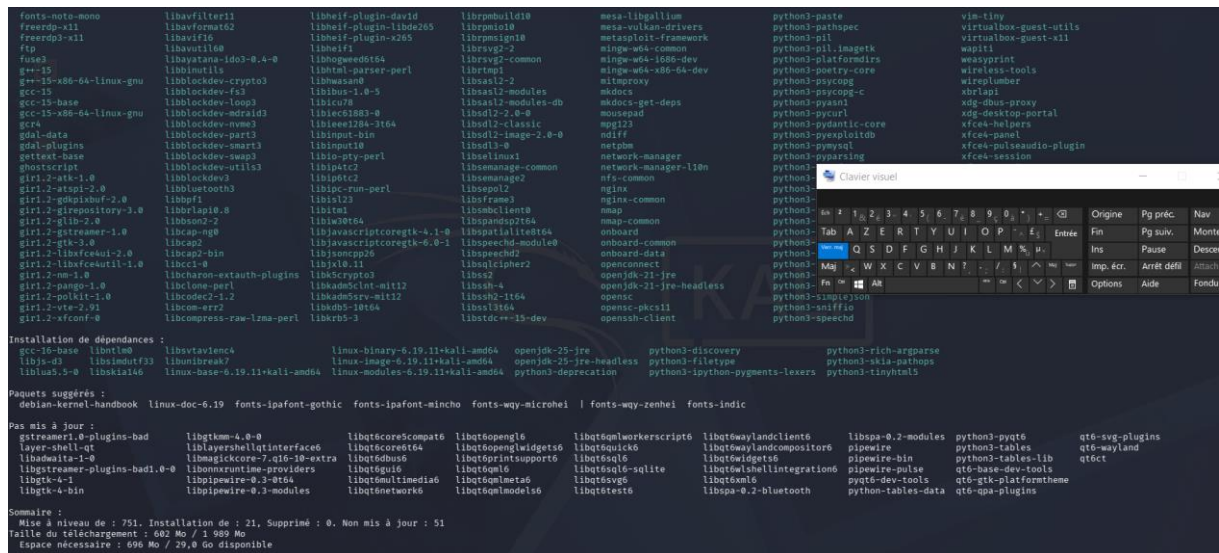
➤ MISE A JOUR DU SYSTEME

Avant toute installation, il est nécessaire de synchroniser les index des paquets locaux avec les dépôts distants. Commande taper **sudo apt update** : cette commande met à jour la liste des paquets disponibles pour s'assurer que nous installons les versions les plus récentes



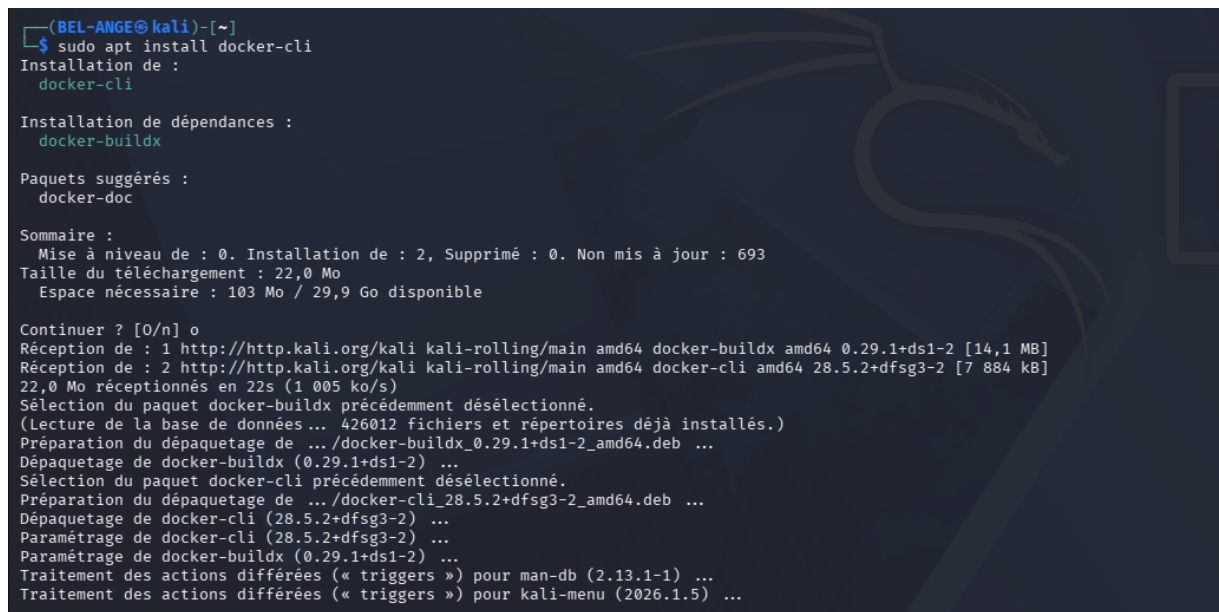
```
(BEL-ANGE@kali)-[~]  
$ sudo apt update  
Atteint : 1 http://http.kali.org/kali kali-rolling InRelease  
802 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.
```

Sudo apt upgrade



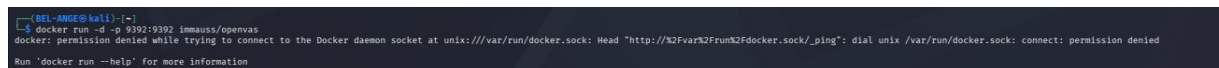
INSTALLATION DE DOCKER CLI

L'étape suivante consiste à installer des outils nécessaires pour utiliser docker en ligne de commande. Nous avons utilisé la commande **sudo apt install docker.io** : elle télécharge et installe le moteur docker et l'interface de ligne de commande

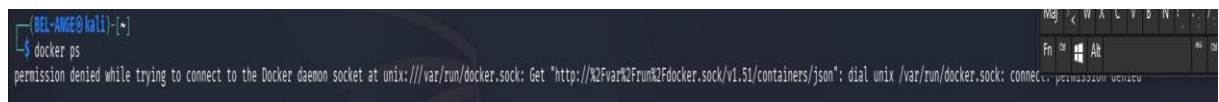


○ Installation et lancement Open VAS

docker run -d -p 9392:9392 immauss/openvas



docker ps



```
(BEL-ANGE@kali)~$ sudo apt install docker.io
Installation de :
docker.io

Installation des dépendances :
containerd criu libcompel1 libintl-perl libintl-xs-perl libmodule-find-perl libproc-processtable-perl libsort-naturally-perl libterm-readkey-perl

Paquets suggérés :
containernetworking-plugins docker-doc btrfs-progs debootstrap rinse rootlesskit xfsprogs zfs-fuse | zfsutils-linux

Sommaire :
Mise à niveau de : 0. Installation de : 15, Supprimé : 0. Non mis à jour : 802
Taille du téléchargement : 55,0 Mo
Espace nécessaire : 226 Mo / 29,8 Go disponible

Continuer ? [O/n] o
Réception de : 1 http://http.kali.org/kali kali-rolling/main amd64 runc amd64 1.3.5+ds1-1 [6 726 kB]
Réception de : 2 http://http.kali.org/kali kali-rolling/main amd64 containerd amd64 2.1.6+ds1-1 [27,8 MB]
Réception de : 3 http://kali.download/kali kali-rolling/main amd64 libcompel1 amd64 4.2-3 [63,1 kB]
Réception de : 4 http://kali.download/kali kali-rolling/main amd64 criu amd64 4.2-3 [556 kB]
Réception de : 5 http://http.kali.org/kali kali-rolling/main amd64 tini-static amd64 0.19.0-6+b1 [281 kB]
Réception de : 6 http://http.kali.org/kali kali-rolling/main amd64 docker.io amd64 23.5.2+dfsg3-2 [18,5 MB]
Réception de : 7 http://kali.download/kali kali-rolling/main amd64 libintl-perl all 1.37-1 [696 kB]
Réception de : 8 http://kali.download/kali kali-rolling/main amd64 libintl-xs-perl amd64 1.37-1 [16,0 kB]
Réception de : 9 http://kali.download/kali kali-rolling/main amd64 libmodule-find-perl all 0.17-1 [10,7 kB]
Réception de : 10 http://http.kali.org/kali kali-rolling/main amd64 libproc-processtable-perl amd64 0.637-1+b1 [42,3 kB]
Réception de : 11 http://kali.download/kali kali-rolling/main amd64 libsort-naturally-perl all 1.03-4 [13,1 kB]
Réception de : 12 http://http.kali.org/kali kali-rolling/main amd64 libterm-readkey-perl amd64 2.38-2+b4 [24,6 kB]
Réception de : 13 http://kali.download/kali kali-rolling/main amd64 needrestart all 3.11-1 [68,6 kB]
Réception de : 14 http://kali.download/kali kali-rolling/main amd64 python3-protobuf amd64 3.21.12-15 [158 kB]
Réception de : 15 http://kali.download/kali kali-rolling/main amd64 python3-pycrui all 4.2-3 [43,2 kB]
55,0 Mo réceptionnés en 38s (1 452 ko/s)
Sélection du paquet runc précédemment désélectionné.
(Lecture de la base de données ... 426275 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../00-runc_1.3.5+ds1-1_amd64.deb ...
Dépaquetage de runc (1.3.5+ds1-1) ...
Sélection du paquet containerd précédemment désélectionné.
Préparation du dépaquetage de .../01-containerd_2.1.6+ds1-1_amd64.deb ...
Dépaquetage de containerd (2.1.6+ds1-1) ...
Sélection du paquet libcompel1:amd64 précédemment désélectionné.
Préparation du dépaquetage de .../02-libcompel1_4.2-3_amd64.deb ...
Dépaquetage de libcompel1:amd64 (4.2-3) ...
```

➤ GESTION DES PRIVILEGES UTILISATEUR

Par défaut, Docker nécessite les droits de super-utilisateur (sudo). Pour permettre à l'utilisateur courant d'exécuter des commandes Docker sans sudo. Nous l'ajoutons au groupe docker. Commande taper **sudo usermod -ag docker \$USER**. -a ajoute l'utilisateur au groupe, -g spécifie le groupe (docker), \$USER représente la variable de l'utilisateur actuel

```
(BEL-ANGE@kali)~$ sudo usermod -ag docker $USER
```

➤ GESTION ET VERIFICATION DU SERVICE Docker

-démarrage du service

Une fois installé, nous devons démarrer le service et vérifier qu'il fonctionne correctement en arrière-plan. commande taper **sudo systemctl start docker** : lance le démon docker sur la machine

```
(BEL-ANGE@kali)~$ sudo systemctl start docker
```

-vérification de l'État

Commande tapée : **sudo systemctl status docker**: permet de confirmer que le service est bien active.

```
(BEL-ANGE@kali)~  
└─$ sudo systemctl status docker  
● docker.service - Docker Application Container Engine  
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)  
   Active: active (running) since Sun 2026-04-26 01:49:03 CEST; 4min 58s ago  
     Invocation: fe32ed1f54304a61817a85b8356d17fc  
   TriggeredBy: ● docker.socket  
       Docs: https://docs.docker.com  
    Main PID: 10691 (dockerd)  
       Tasks: 8  
     Memory: 57.6M (peak: 57.8M)  
        CPU: 820ms  
     CGroup: /system.slice/docker.service  
             └─10691 /usr/sbin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock  
  
avril 26 01:48:58 kali dockerd[10691]: time="2026-04-26T01:48:58.878515270+02:00" level=info msg="COI directory does not exist, skipping: failed to monitor for changes: no such file or directory" dir=/etc/cdi  
avril 26 01:48:59 kali dockerd[10691]: time="2026-04-26T01:48:59.578673821+02:00" level=info msg="Creating a containerd client" address=/run/containerd/containerd.sock timeout=1m0s  
avril 26 01:49:00 kali dockerd[10691]: time="2026-04-26T01:49:00.295619334+02:00" level=info msg="Loading containers: start."  
avril 26 01:49:02 kali dockerd[10691]: time="2026-04-26T01:49:02.588565487+02:00" level=info msg="Loading containers: done."  
avril 26 01:49:02 kali dockerd[10691]: time="2026-04-26T01:49:02.904119964+02:00" level=info msg="Docker daemon" commit=3ef3c07c883be49a2e8097bcf6d2952fce1d70eb containerd-snapshotter=false storage-driver=overlay2 version=20.5.2+dfsg3  
avril 26 01:49:02 kali dockerd[10691]: time="2026-04-26T01:49:02.904274768+02:00" level=info msg="Initializing buildkit"  
avril 26 01:49:03 kali dockerd[10691]: time="2026-04-26T01:49:03.298318454+02:00" level=info msg="Completed buildkit initialization"  
avril 26 01:49:03 kali dockerd[10691]: time="2026-04-26T01:49:03.318442409+02:00" level=info msg="Daemon has completed initialization"  
avril 26 01:49:03 kali dockerd[10691]: time="2026-04-26T01:49:03.318778804+02:00" level=info msg="API listen on /run/docker.sock"  
avril 26 01:49:03 kali systemd[1]: Started docker.service - Docker Application Container Engine.
```

➤ DEPLOIEMENT D'UN CONTENEUR

Nous testons l'installation en lançant un conteneur en mode détaché. Commande tapée **sudo docker run -d -p 9392.9392 imauss/openvas** : permet au conteneur de s'exécuter en arrière-plan

```
(BEL-ANGE@kali)~  
└─$ docker run -d -p 9392:9392 imauss/openvas  
docker: permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Head "http://%2Fvar%2Frun%2Fdocker.sock/_ping": dial unix /var/run/docker.sock: connect: permission denied  
  
Run 'docker run --help' for more information
```

➤ SECURITER ET PARE FEU

Pour sécuriser l'hôte, nous vérifions l'état du pare feu système en tapant la commande **sudo ufw status verbose** : qui affiche l'Etat détaillé du pare-feu, notamment les règles actifs et les ports ouverts

```

bel-ange@bel-ange:~$ sudo ufw status verbose
État : actif
Journalisation : on (low)
Par défaut : deny (incoming), allow (outgoing), disabled (routed)
Nouveaux profils : skip

Vers          Action      De
----          -
22/tcp        ALLOW IN    Anywhere
80/tcp        ALLOW IN    Anywhere
22/tcp (v6)   ALLOW IN    Anywhere (v6)
80/tcp (v6)   ALLOW IN    Anywhere (v6)

```

3. Analyse des vulnérabilités : CVE, impact, exploitation

🚧 Exemple 1

- Nom :
- CVE :
- Sévérité :
- Description :
- Impact :
- Exploitation possible :

🚧 Exemple 2

- Nom :
- CVE :
- Sévérité :
- Description :
- Impact :
- Exploitation possible :

✓ Types de vulnérabilités observées

- Mauvaise configuration
- Logiciels obsolètes
- Ports ouverts
- Failles réseau

4. Corrections (AVEC CAPTURES)

○ Mise à jour système

```
sudo apt update && sudo apt upgrade
```

```

(BEL-ANGE@kali)~$ sudo apt update
Atteint : 1 http://http.kali.org/kali kali-rolling InRelease
802 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.

```


- **Sécurisation SSH**

`sudo nano /etc/ssh/sshd_config`

```
(BEL-ANGE@kali)-[~]  
$ sudo nano /etc/ssh/sshd_config  
(BEL-ANGE@kali)-[~]
```

La commande m'a renvoyé ici

```
# This is the sshd server system-wide configuration file. See  
# sshd_config(5) for more information.  
  
# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games  
  
# The strategy used for options in the default sshd_config shipped with  
# OpenSSH is to specify options with their default value where  
# possible, but leave them commented. Uncommented options override the  
# default value.  
  
Include /etc/ssh/sshd_config.d/*.conf  
  
#Port 22  
#AddressFamily any  
#ListenAddress 0.0.0.0  
#ListenAddress ::  
  
#HostKey /etc/ssh/ssh_host_rsa_key  
#HostKey /etc/ssh/ssh_host_ecdsa_key  
#HostKey /etc/ssh/ssh_host_ed25519_key  
  
# Ciphers and keying  
#RekeyLimit default none  
  
# Logging  
#SyslogFacility AUTH  
#LogLevel INFO  
  
# Authentication:  
  
#LoginGraceTime 2m  
#PermitRootLogin prohibit-password  
#StrictModes yes  
#MaxAuthTries 6  
#MaxSessions 10  
  
#PubkeyAuthentication yes  
  
# Expect .ssh/authorized_keys2 to be disregarded by default in future.  
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2  
  
#AuthorizedPrincipalsFile none  
  
#AuthorizedKeysCommand none  
#AuthorizedKeysCommandUser nobody  
  
# For this to work you will also need host keys in /etc/ssh/ssh_known_hosts
```

- Désactiver root login

```
#LoginGraceTime 2m  
PermitRootLogin no  
#StrictModes yes
```

5. Comparaison Nessus et Open VAS

Critère	Nessus	Open VAS
Licence	Propriétaire	Open Source
Facilité	Très facile	Moyenne
Performance	Élevée	Bonne
Entreprise	Standard	Alternative
Gratuit	Non	Oui

Analyse

Nessus est plus simple et rapide, Open VAS demander plus de configuration mais reste efficace

6. Analyse professionnelle

Dans un contexte professionnel :

- Nessus :
 - Gain de temps
 - Support technique
 - Déploiement rapide
- Open VAS :
 - Gratuit
 - Flexible
 - Nécessite expertise

Open VAS est adapté :

- aux PME
- aux environnements de test
- ✓ Nessus est recommandé :
 - en entreprise critique
 - pour audits réguliers

7. Réponses

Quel outil est le plus adapté en entreprise ?

Nessus est le plus adapté car son interface est simple, il a un gain de temps, un support officiel et une meilleure intégration en entreprise

Open VAS reste pertinent si son budget est limité et si il y a une équipe technique compétente.

Conclusion

Ce TP m'a permis d'utiliser un scanner de vulnérabilités, Comprendre les failles systèmes et Apprendre à corriger les vulnérabilités. Open VAS constituer une solution efficace mais plus technique, tandis que Nessus reste la référence en entreprise.

